

# 数字硬件设计实验报告

---

## 实验 2：组合逻辑部件设计

---

实验环境： Logisim 2.16

实验内容： 译码器、编码器、加法器、汉明码校验、桶形移位器

2026 年 4 月 18 日

## 零、目录

## 一、实验整体方案设计

本次实验共包含五个子实验，依次为：3-8 译码器、8-3 优先级编码器、4 位串行进位加法器（含补码减法）、7 位汉明码校验电路以及 8 位桶形移位器。各子实验相互独立，分别保存为独立的 Logisim 文件（lab2.1.circ ~ lab2.5.circ）。

### 1.1 各子模块输入输出定义

表 1: 各子模块引脚定义总表

| 实验  | 模块名        | 输入引脚                                                          | 输出引脚                                             |
|-----|------------|---------------------------------------------------------------|--------------------------------------------------|
| 2.1 | 3-8 译码器    | G1(1 位), G2A_L(1 位),<br>G2B_L(1 位), A(1 位),<br>B(1 位), C(1 位) | Y0_L ~ Y7_L (各 1 位,<br>低有效)                      |
| 2.2 | 8-3 优先级编码器 | I0 ~ I7 (各 1 位)                                               | O0, O1, O2 (各 1 位); 16<br>进制数码管显示                |
| 2.3 | 4 位串行加减法器  | X[3:0] (4 位), Y[3:0] (4 位),<br>Cin (1 位)                      | SUM[3:0] (4 位), Cout (1<br>位); 16 进制数码管显示        |
| 2.4 | 7 位汉明码纠错   | Input[6:0] (7 位, 含数据位<br>和校验位)                                | Output[6:0] (纠错后输出),<br>NoError (1 位), 错误位<br>编号 |
| 2.5 | 8 位桶形移位器   | Din[7:0] (8 位), shamt[2:0]<br>(3 位), L/R (1 位), A/L<br>(1 位)  | Dout[7:0] (8 位)                                  |

### 1.2 实验 2.3 顶层模块图

实验 2.3（4 位串行进位加法器）是本次实验中唯一包含多层子电路的模块，其顶层结构由 4 个全加器子电路（FA）串联而成，如图1所示。

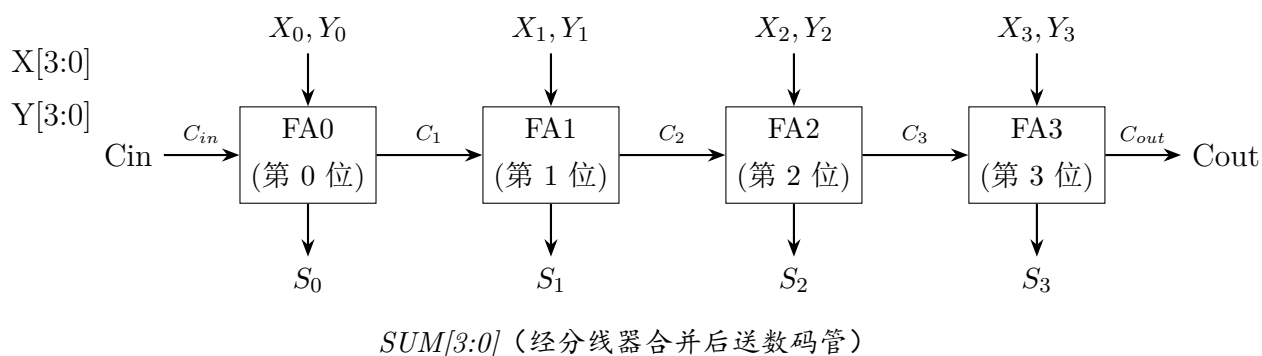


图 1: 4 位串行进位加法器顶层模块图

全加器子电路 FA 的引脚定义如下:

- 输入: A (1 位加数)、B (1 位被加数)、Cin (1 位低位进位)
- 输出: F (1 位和)、Cout (1 位进位输出)

补码减法的实现方式: 当 Cin=1 时, 对 Y 的每一位接入异或门与 Cin 进行异或, 将 Y 取反后连同 Cin=1 送入最低位全加器, 从而实现  $F = X + \bar{Y} + 1 = X - Y$  (补码减法)。

## 二、实验原理图与电路图

### 2.1 实验 2.1: 3-8 译码器

#### 2.1.1 输入输出引脚定义

- 输入: 使能端 G1 (高有效)、G2A\_L (低有效)、G2B\_L (低有效); 地址端 C (高位)、B、A (低位)
- 输出: Y0\_L ~ Y7\_L, 共 8 路, 低电平有效

使能条件: 仅当  $G1 = 1$ ,  $G2A\_L = 0$ ,  $G2B\_L = 0$  时, 译码器正常工作; 否则所有输出为高 (禁用)。

#### 2.1.2 电路实现说明

根据 74X138 原理图, 使用 8 个 4 输入与非门、7 个非门、1 个与门构成。三个地址输入 A、B、C 各自经非门产生其反变量, 再与相应的正变量组合, 连同使能信号一起接入对应的 4 输入与非门, 输出即为低有效的译码结果。

### 2.1.3 Logisim 电路图

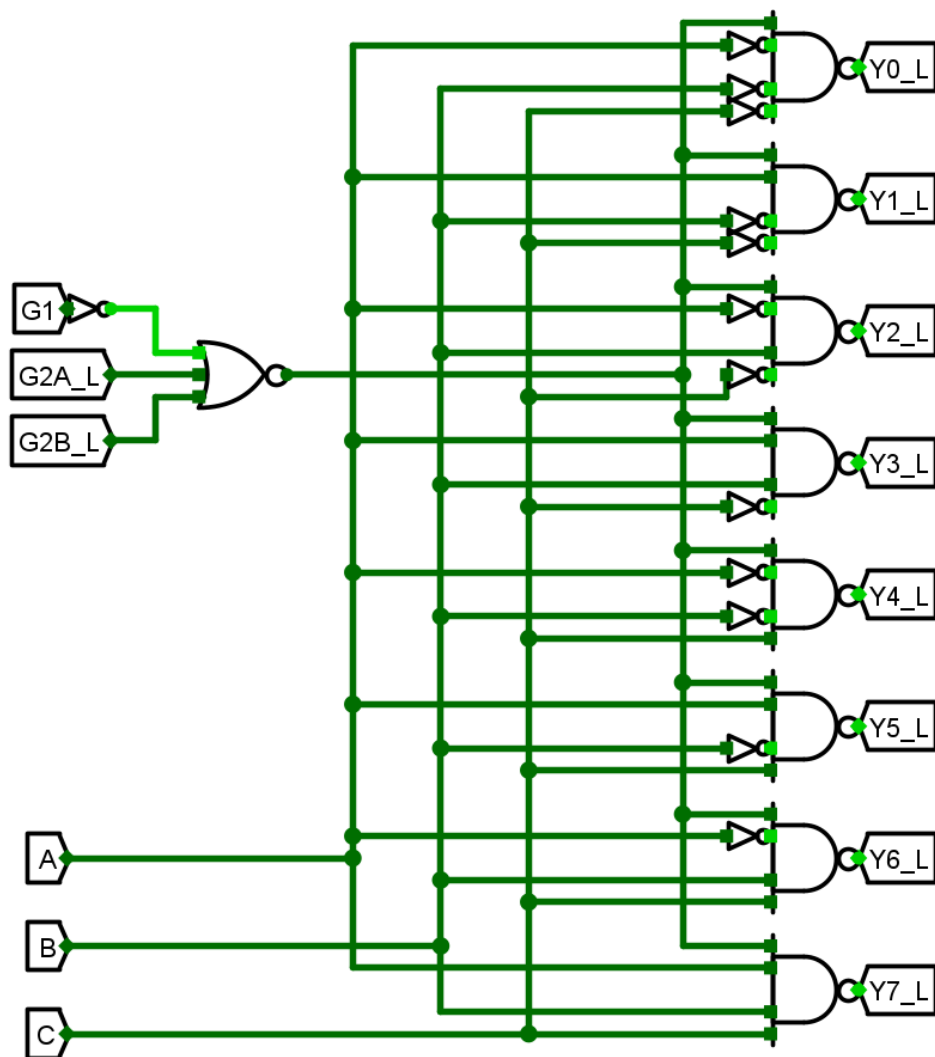


图 2: 电路图

## 2.2 实验 2.2: 8-3 优先级编码器

### 2.2.1 输入输出引脚定义

- 输入:  $I_0 \sim I_7$ , 共 8 路, 高电平有效;  $I_7$  优先级最高
- 输出:  $O_2 O_1 O_0$  (3 位二进制编码); 连接 16 进制数码管显示结果

### 2.2.2 电路实现说明

优先级编码器的逻辑表达式由原理图推导而来。以  $O_2$  为例:

$$O_2 = I_4 + I_5 + I_6 + I_7$$

其余各位类似，每个输出由一个 4 输入或门（或等效与门网络）实现。输入引脚设置为反转（低有效输入），与原理图保持一致。输出通过分线器合成 3 位总线后接入 16 进制数码管。

### 2.2.3 Logisim 电路图

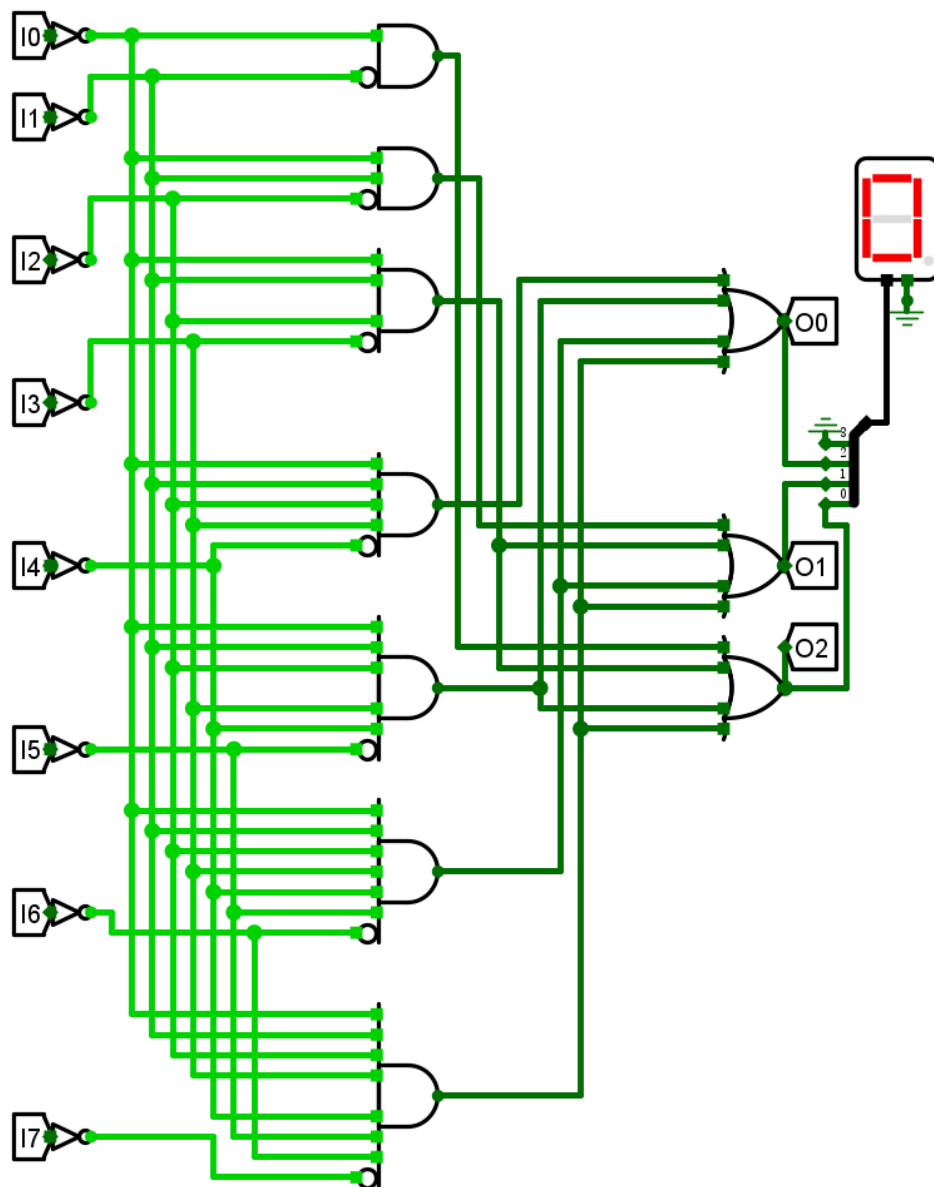


图 3: 电路图

2.3 实验 2.3：4 位串行进位加法器与补码减法

2.3.1 全加器子电路（FA）原理图

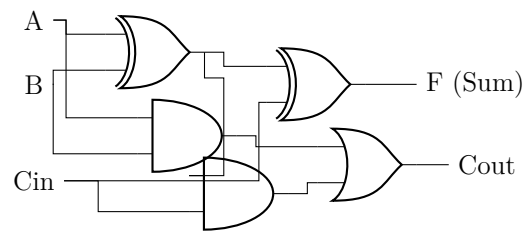


图 4: 全加器（FA）逻辑原理图

进位输出可用与非-与非电路等效替换与或电路，实现上使用两级与非门：

$$C_{out} = \overline{AB \cdot (A \oplus B) \cdot C_{in}}$$

2.3.2 4 位串行进位加法器与补码减法原理图

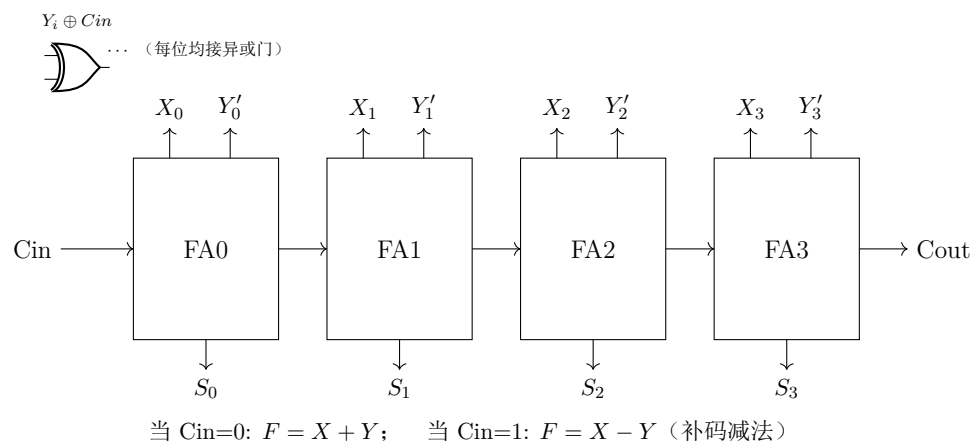


图 5: 4 位串行进位加减法器原理图

### 2.3.3 Logisim 电路图

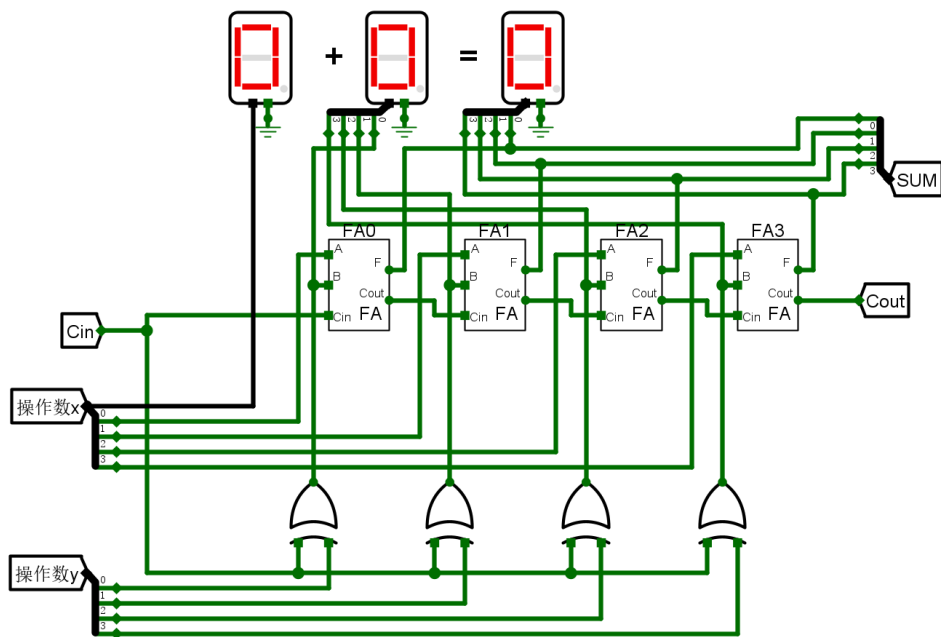


图 6: 电路图

## 2.4 实验 2.4: 7 位汉明码校验电路

### 2.4.1 输入输出引脚定义

- 输入: Input[6:0], 位排列为  $P_1P_2M_1P_3M_2M_3M_4$  (位 1~ 位 7)
- 输出: Output[6:0] (纠错后的 7 位码字), NoError (1 位, 无错时为 1), 错误位编号 (显示故障字  $S_3S_2S_1$ )

### 2.4.2 汉明码校验原理

故障字各位计算公式 (偶校验):

$$S_1 = M_1 \oplus M_2 \oplus M_4 \oplus P_1 \quad (1)$$

$$S_2 = M_1 \oplus M_3 \oplus M_4 \oplus P_2 \quad (2)$$

$$S_3 = M_2 \oplus M_3 \oplus M_4 \oplus P_3 \quad (3)$$

故障字  $S_3S_2S_1$  的值即为出错位在码字中的位置编号 (0 表示无错)。纠错时将该位取反即可。

### 2.4.3 电路实现说明

电路由三部分组成:



1. 奇偶校验子电路: 3 个 4 输入奇偶校验器 (异或链), 分别计算  $S_1$ 、 $S_2$ 、 $S_3$ ;
2. 3-8 译码器: 以  $S_3S_2S_1$  为地址, 输出低有效的  $Y_0 \sim Y_7$ , 经取反后得到对应位的纠错使能信号;
3. 异或纠错门: 每个数据/校验位与对应的译码器输出 (已取反为高有效) 异或, 实现单比特翻转纠错。

#### 2.4.4 Logisim 电路图

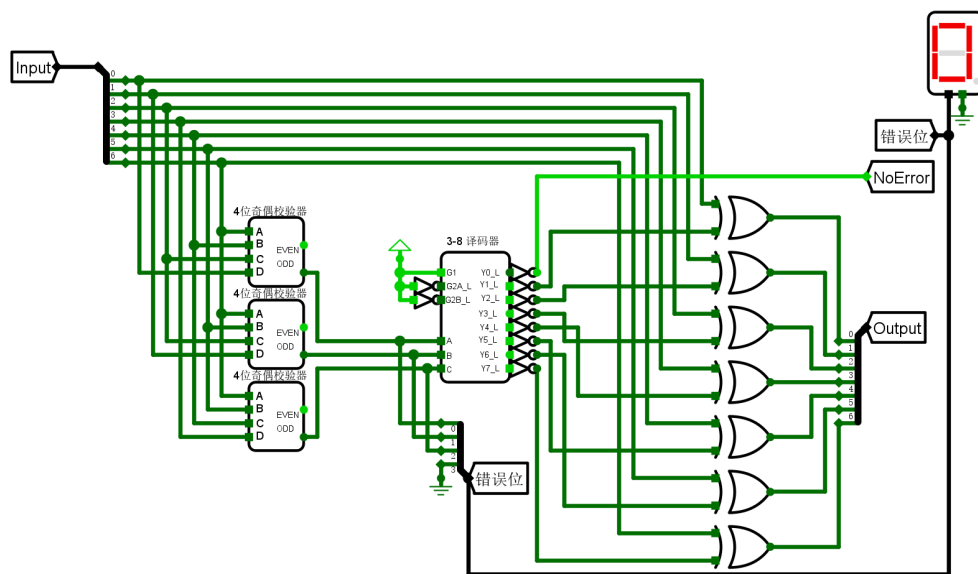


图 7: 电路图

### 2.5 实验 2.5: 8 位桶形移位器

#### 2.5.1 输入输出引脚定义

- 输入:  $Din[7:0]$  (8 位数据),  $shamt[2:0]$  (移位位数, 0~7), L/R (方向: 1= 左移, 0= 右移), A/L (类型: 1= 算术/循环, 0= 逻辑)
- 输出:  $Dout[7:0]$  (移位结果)

## 2.5.2 三级多路选择器原理图

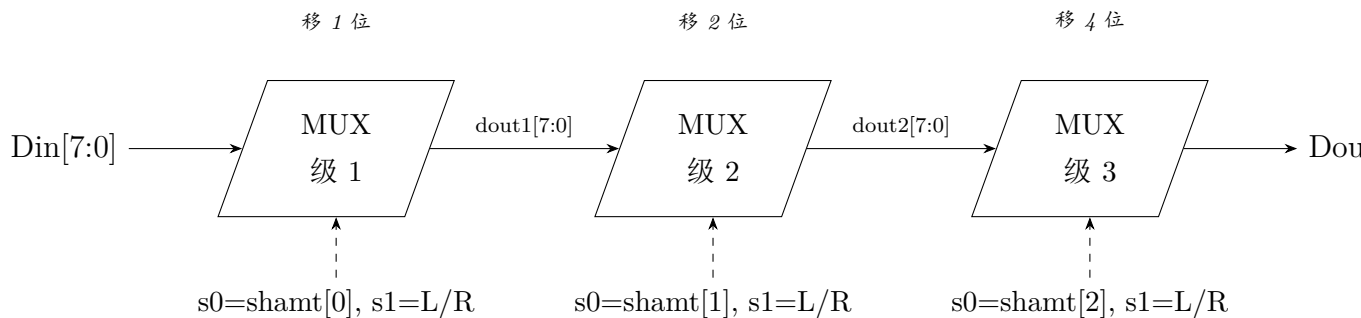


图 8: 8 位桶形移位器三级 MUX 原理图

每级 4 路 8 位选择器的选择逻辑如下：

- $s_0 = 0$ （不移位）：直接传递当前数据；
- $s_0 = 1, s_1 = 0$ （右移）：数据右移对应位数，高位补 0（逻辑）或符号位  $\text{din}[7]$ （算术）；
- $s_0 = 1, s_1 = 1$ （左移）：数据左移对应位数，低位补 0（逻辑）或移出位（循环）。

## 2.5.3 Logisim 电路图

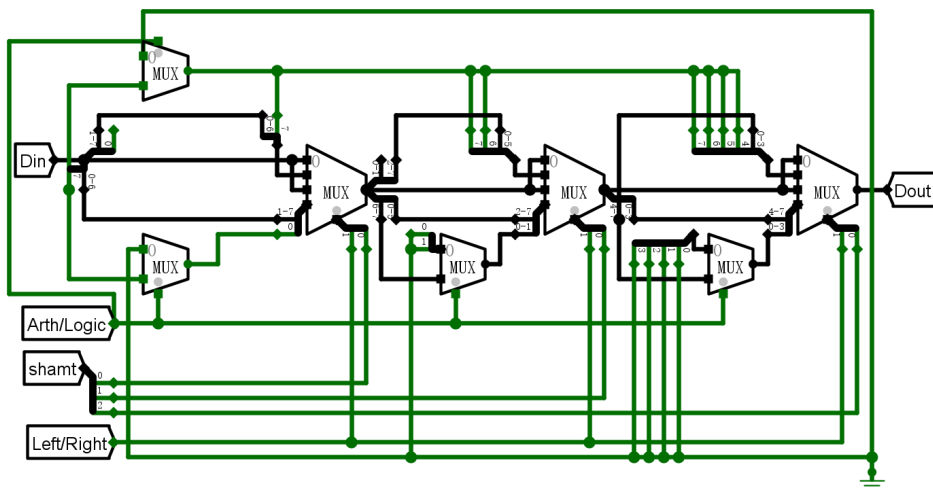


图 9: 电路图

三、实验数据仿真测试

3.1 3-8 译码器功能表（实验 2.1）

表 2: 74X138 功能仿真结果

| G1 | G2A_L | G2B_L | C | B | A | Y7 | Y6 | Y5 | Y4 | Y3 | Y2 | Y1 | Y0 |
|----|-------|-------|---|---|---|----|----|----|----|----|----|----|----|
| X  | 1     | X     | X | X | X | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1  |
| X  | X     | 1     | X | X | X | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1  |
| 0  | X     | X     | X | X | X | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1  |
| 1  | 0     | 0     | 0 | 0 | 0 | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 0  |
| 1  | 0     | 0     | 0 | 0 | 1 | 1  | 1  | 1  | 1  | 1  | 1  | 0  | 1  |
| 1  | 0     | 0     | 0 | 1 | 0 | 1  | 1  | 1  | 1  | 1  | 0  | 1  | 1  |
| 1  | 0     | 0     | 0 | 1 | 1 | 1  | 1  | 1  | 1  | 0  | 1  | 1  | 1  |
| 1  | 0     | 0     | 1 | 0 | 0 | 1  | 1  | 1  | 0  | 1  | 1  | 1  | 1  |
| 1  | 0     | 0     | 1 | 0 | 1 | 1  | 1  | 0  | 1  | 1  | 1  | 1  | 1  |
| 1  | 0     | 0     | 1 | 1 | 0 | 1  | 0  | 1  | 1  | 1  | 1  | 1  | 1  |
| 1  | 0     | 0     | 1 | 1 | 1 | 0  | 1  | 1  | 1  | 1  | 1  | 1  | 1  |

分析：当使能条件满足时，仅有与地址 CBA 对应的输出端为低电平（0），其余均为高电平（1），符合 74X138 低有效输出的设计规范。

3.2 8-3 优先级编码器功能表（实验 2.2）

表 3: 8-3 优先级编码器功能仿真结果

| I0 | I1 | I2 | I3 | I4 | I5 | I6 | I7 | O2 | O1 | O0 | Hex 显示 |
|----|----|----|----|----|----|----|----|----|----|----|--------|
| 1  | X  | X  | X  | X  | X  | X  | X  | 0  | 0  | 0  | 0      |
| 0  | 1  | X  | X  | X  | X  | X  | X  | 0  | 0  | 1  | 1      |
| 0  | 0  | 1  | X  | X  | X  | X  | X  | 0  | 1  | 0  | 2      |
| 0  | 0  | 0  | 1  | X  | X  | X  | X  | 0  | 1  | 1  | 3      |
| 0  | 0  | 0  | 0  | 1  | X  | X  | X  | 1  | 0  | 0  | 4      |
| 0  | 0  | 0  | 0  | 0  | 1  | X  | X  | 1  | 0  | 1  | 5      |
| 0  | 0  | 0  | 0  | 0  | 0  | 1  | X  | 1  | 1  | 0  | 6      |
| 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 1  | 1  | 1  | 7      |

分析: 优先级从  $I_0$  (最低) 到  $I_7$  (最高), 高优先级输入有效时屏蔽低优先级, 输出编码与数码管显示一致, 验证了优先级编码逻辑的正确性。

3.3 4 位串行进位加法器测试 (实验 2.3)

表 4: 4 位加减法器功能仿真结果 (部分测试用例)

| X (十进制) | Y (十进制) | Cin | SUM (十进制) | Cout | 说明                           |
|---------|---------|-----|-----------|------|------------------------------|
| 5       | 3       | 0   | 8         | 0    | $5 + 3 = 8$ , 无溢出            |
| 9       | 7       | 0   | 0         | 1    | $9 + 7 = 16$ , 产生进位          |
| 7       | 3       | 1   | 4         | 1    | $7 - 3 = 4$ , 补码减法           |
| 3       | 7       | 1   | 12        | 0    | $3 - 7 = -4$ , 结果为负 (补码为 12) |
| 15      | 1       | 0   | 0         | 1    | $15 + 1 = 16$ , 进位           |
| 0       | 0       | 0   | 0         | 0    | 全零                           |

分析: 加法结果与预期吻合。减法通过补码加法实现,  $X - Y = X + \overline{Y} + 1$ , 当结果为负时以补码形式呈现, Cout=0 表示借位, 符合有符号数运算规则。

3.4 7 位汉明码纠错测试 (实验 2.4)

表 5: 7 位汉明码纠错仿真测试

| 输入码字 (位 7~1) |       |       |       |       |       |       | 故障字   |       |       | 错误位 | NoError | 输出是否正确 |
|--------------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-----|---------|--------|
| $P_1$        | $P_2$ | $M_1$ | $P_3$ | $M_2$ | $M_3$ | $M_4$ | $S_1$ | $S_2$ | $S_3$ |     |         |        |
| 0            | 1     | 1     | 0     | 0     | 1     | 1     | 0     | 0     | 0     | 无错  | 1       | ✓      |
| 1            | 1     | 1     | 0     | 0     | 1     | 1     | 1     | 0     | 0     | 位 1 | 0       | ✓      |
| 0            | 0     | 1     | 0     | 0     | 1     | 1     | 0     | 1     | 0     | 位 2 | 0       | ✓      |
| 0            | 1     | 0     | 0     | 0     | 1     | 1     | 1     | 1     | 0     | 位 3 | 0       | ✓      |
| 0            | 1     | 1     | 1     | 0     | 1     | 1     | 0     | 0     | 1     | 位 4 | 0       | ✓      |
| 0            | 1     | 1     | 0     | 1     | 1     | 1     | 1     | 0     | 1     | 位 5 | 0       | ✓      |

分析: 电路能正确计算故障字, 精确定位出错位, 并将该位取反完成纠错。无错时 NoError=1, 有错时 NoError=0 并给出错误位编号, 验证了汉明码单比特纠错功能。注意 3-8 译码器输出低有效, 需经非门取反后再驱动异或纠错门, 实验中已处理此细节。

### 3.5 8 位桶形移位器测试 (实验 2.5)

表 6: 8 位桶形移位器功能仿真结果 (Din=10110100)

| L/R | A/L | shamt | Dout (二进制) | 说明                |
|-----|-----|-------|------------|-------------------|
| 0   | 0   | 000   | 10110100   | 不移位               |
| 0   | 0   | 010   | 00101101   | 逻辑右移 2 位          |
| 0   | 1   | 010   | 11101101   | 算术右移 2 位 (符号位补 1) |
| 1   | 0   | 010   | 11010000   | 逻辑左移 2 位          |
| 1   | 1   | 010   | 11010010   | 循环左移 2 位          |
| 0   | 0   | 111   | 00000001   | 逻辑右移 7 位          |
| 1   | 0   | 111   | 00000000   | 逻辑左移 7 位          |

**分析:** 三级 MUX 结构能在组合逻辑时延内一次完成 0~7 位的任意移位, 逻辑/算术/循环三种类型均验证正确。循环右移通过循环左移互补位数实现 ( $n$  位循环右移 =  $8 - n$  位循环左移), 与实验设计一致。

## 四、思考题解析

### 4.1 思考题 1: 带标志位的 4 位加法器

在 4 位串行进位加法器基础上, 增加以下组合逻辑生成标志位:

- **进位标志 CF** (Carry Flag): 直接取最高位全加器 FA3 的进位输出 Cout:

$$CF = C_{out}$$

- **溢出标志 OF** (Overflow Flag): 有符号数溢出, 当最高位进位与次高位进位不一致时溢出:

$$OF = C_{out} \oplus C_3$$

其中  $C_3$  为 FA3 的进位输入 (即 FA2 的进位输出)。

- **符号标志 SF** (Sign Flag): 结果的最高位 (符号位):

$$SF = S_3$$

- **零标志 ZF** (Zero Flag): 结果全为 0 时置 1, 可用 4 输入或非门实现:

$$ZF = \overline{S_3 + S_2 + S_1 + S_0} = \overline{S_3} \cdot \overline{S_2} \cdot \overline{S_1} \cdot \overline{S_0}$$

在 Logisim 中可用 4 输入 NOR 门 (或等效的 OR+NOT) 实现。

#### 4.2 思考题 2: 通过标志位输出大于/小于信号

利用上述标志位, 可以组合出有符号数比较结果:

- $X > Y$  (有符号): 在  $X - Y$  的结果中,  $X > Y$  等价于结果大于 0 且无溢出。  
结合标志位:

$$X > Y \Leftrightarrow \overline{ZF} \wedge (\overline{SF \oplus OF})$$

即: 结果非零, 且符号位与溢出标志同号 (等价于差值为正)。

- $X < Y$  (有符号):

$$X < Y \Leftrightarrow SF \oplus OF$$

即: 符号位与溢出标志不同 (等价于差值为负)。

在 Logisim 中, 将加法器 Cin 接 1 (执行减法  $X - Y$ ), 再将 SF、OF、ZF 按上述逻辑连接异或门和与门即可输出比较信号。

#### 4.3 思考题 3: 4 位无符号二进制数比较器 (不使用加法器)

对于无符号 4 位数  $X = X_3X_2X_1X_0$  与  $Y = Y_3Y_2Y_1Y_0$ , 从最高位向最低位逐位比较:

大于 ( $X > Y$ ) 的逻辑表达式:

$$X > Y = X_3\overline{Y_3} + E_3X_2\overline{Y_2} + E_3E_2X_1\overline{Y_1} + E_3E_2E_1X_0\overline{Y_0}$$

其中  $E_i = \overline{X_i \oplus Y_i}$  (第  $i$  位相等标志)。

小于 ( $X < Y$ ) 的逻辑表达式:

$$X < Y = \overline{Y_3}X_3 \text{ 替换为 } \overline{X_3}Y_3 + E_3\overline{X_2}Y_2 + E_3E_2\overline{X_1}Y_1 + E_3E_2E_1\overline{X_0}Y_0$$

在 Logisim 中可用非门、与门、或门、异或门按上述表达式直接连线实现, 无需任何加法器组件。

#### 4.4 思考题 4: 8 位桶形移位器扩展到 32 位

8 位桶形移位器使用 3 级 MUX (每级分别处理移位量 1、2、4 位) 实现任意 0~7 位移位。

将其扩展到 32 位, 需要将 shamt 从 3 位扩展到 5 位 (因为  $\lceil \log_2 32 \rceil = 5$ ), 并增加到 5 级 MUX, 每级对应的移位量分别为 1、2、4、8、16 位, 共需 5 组 32 位宽的 4 路选择器。

具体扩展方案如下:

1. **数据宽度:** 将所有多路选择器和分线器的数据宽度从 8 位改为 32 位;
2. **移位级数:** 增加第 4 级 (移 8 位, 由 `shamt[3]` 控制) 和第 5 级 (移 16 位, 由 `shamt[4]` 控制);
3. **填充位处理:**
  - 逻辑移位时, 移入位为 0;
  - 算术右移时, 移入位为 `din[31]` (符号位);
  - 循环移位时, 移出位循环填入;
4. **模块化复用:** 可将 8 位桶形移位器作为子电路, 将 4 个 8 位单元组合并配合适当的位拼接逻辑构成 32 位版本, 也可直接参照 8 位设计逐步扩展到 32 位。

以上扩展方案的时延仅增加 2 级 MUX 延迟 (从 3 级增至 5 级), 相比串行移位器 (最多 31 级) 具有显著的速度优势, 体现了桶形移位器的设计价值。